ADR-0003 · Decisiones del Sprint 2 (pagos y saldo a favor)

Fecha: 2026-10-29 · Estado: propuesta (pendiente de aprobación)

Alcance: **HU04 (P0)** — pagos parciales o totales con saldo exacto, sin pagar más de lo adeudado y con control de concurrencia — y **HU09 (P1)** — anulación de un pago con motivo, el saldo vuelve a subir y queda auditado. Se suman el **saldo a favor del cliente** y la **idempotencia**, decididos con el responsable del producto. Cierra el pendiente del ADR-0002.

> Las historias están en `docs/requirements/historias-de-usuario.md`. La *Especificación MVP v1.0* completa todavía no está versionada. Los puntos #6, #7, #12 a #17 y #19 se revisaron y decidieron con el responsable del producto el 2026-10-28.

Decisiones

Pagos

#	Decisión	Motivo
1	Se usa la tabla `payments` creada en `0001`. `0002` le agrega `idempotency_key` (UUID, único, opcional) y `0003` crea `client_credits` (#11). `voided_by` no se agrega: quién anuló queda en `audit_events`.	El ADR-0002 (#1) ya dejó lista la tabla de pagos.
2	Endpoints `GET/POST /payments`, `GET /payments/{id}` y `POST /payments/{id}/void`. El historial de una factura es `GET /payments?invoice_id=…`.	Un solo recurso con filtros, igual que `/invoices`. Anular es una acción con motivo obligatorio, no un `PATCH` genérico.
3	**Registrar**: cualquier usuario autenticado. **Anular**: solo administrador (403 para el operador).	HU09 y §4: "anulación lógica de pago por administrador".
4	**Concurrencia**: toda operación de cobro hace `SELECT … FOR UPDATE` en el orden **cliente** → **factura** → **pago**.	HU04 y §16 multiusuario. Con la factura bloqueada, dos cobros simultáneos nunca superan el importe; con el cliente bloqueado, el saldo a favor nunca se gasta dos veces. Al bloquear siempre en el mismo orden no hay deadlocks (la edición de facturas del ADR-0002 #13 solo bloquea la factura).
5	Los tests de concurrencia usan **dos conexiones reales con COMMIT** (`test_payments_concurrency.py`). Se verificó que **fallan si se quita el bloqueo** de la factura o del cliente.	El resto de los tests corre dentro de una transacción que se revierte y no puede reproducir dos sesiones compitiendo.
6	**Fecha de pago** = fecha de **recepción**: es opcional (por defecto, hoy en Buenos Aires) y **no puede ser posterior a hoy** (422, `field: payment_date`). **Sí puede ser anterior a la emisión** de la factura.	Cheques diferidos y e-cheq se registran el día que se reciben; si rebotan, el administrador anula el pago con motivo. Se permiten las fechas anteriores a la emisión para reflejar anticipos reales.

#	Decisión	Motivo
7	**Medio de pago** con lista cerrada (confirmada): `efectivo`, `transferencia`, `cheque`, `tarjeta`, `deposito`, `otro`. Se valida en la API, sin CHECK en la base. Existe además el método interno `saldo_a_favor` (#14), que no se puede elegir al registrar un cobro.	La columna `method VARCHAR(30)` de §7 no define valores. Si la lista cambia, se modifica el enum sin migrar.
8	`reference` opcional (hasta 100 caracteres; vacío → `null`).	Nº de transferencia, cheque, etc.
9	**Anulación de pagos**: motivo obligatorio (3 a 500 caracteres, se recortan espacios); guarda `is_void`, `void_reason` y `voided_at` (UTC). **No se puede revertir** ni anular dos veces (409 `payment_already_void`).	HU09: el pago no se borra y queda quién lo anuló (`audit_events`), cuándo (`voided_at`) y por qué (`void_reason`). Coherente con la anulación de facturas (ADR-0002 #12). Para corregir un pago mal anulado, se registra uno nuevo.
10	No se registran pagos sobre una **factura anulada** (409 `invoice_void`). Sí se aceptan pagos de un **cliente inactivo**.	Desactivar un cliente impide facturarle (ADR-0002 #14), pero no cobrar la deuda que ya tiene.

Saldo a favor del cliente

#	Decisión	Motivo
11	Tabla `client_credits`: cada fila es dinero recibido que todavía no se imputó. Puede ser el **excedente** de un pago (`payment_id` apunta al pago que lo generó) o un **cobro a cuenta** (`payment_id` NULL).	Los pagos siguen siendo "lo imputado a una factura", así que el cálculo de saldo y estado de facturas (ADR-0002 #10) no cambia.
12	**Sobrepago con confirmación**: si `amount > saldo`, se rechaza con 409 `amount_exceeds_balance` (el mensaje informa saldo y excedente) **salvo** `allow_credit: true`. Con la confirmación, el pago imputa solo el saldo y el excedente queda como saldo a favor. La respuesta informa `amount` (lo imputado) y `credit_amount` (el excedente); lo recibido es la suma. Pagar una factura ya saldada da 409 `invoice_already_paid` aunque se confirme.	Cumple HU04: sin confirmación el sobrepago se rechaza, y aun confirmado **la factura nunca recibe más de lo adeudado** (el excedente va al cliente, no a la factura). Una transferencia de $125.100 por una factura de $125.000 se registra sin que la caja deje de coincidir con el banco. Pedir confirmación evita que un error de tipeo ($1.250.000) genere un crédito enorme.
13	**Cobro a cuenta**: `POST /credits` (cliente, importe, medio, fecha, referencia) registra un anticipo sin factura. Las fechas siguen la regla #6. No se acepta para un **cliente inactivo** (409 `client_inactive`).	Cubre los anticipos que se cobran antes de emitir la factura. A un cliente inactivo no se le puede facturar, así que un anticipo no tendría dónde imputarse.
14	**Imputación manual**: `POST /credits/apply` (factura, importe) la hace el operador y registra un pago con método `saldo_a_favor` y fecha de hoy. El importe no puede superar ni el saldo de la factura (`amount_exceeds_balance`) ni el disponible del cliente (409 `insufficient_credit`). Solo se usa crédito del mismo cliente de la factura.	El operador imputa lo que el cliente indica; nada se aplica solo.

#	Decisión	Motivo
15	**Disponible** = créditos vigentes − pagos vigentes `saldo_a_favor` del cliente. Se calcula, no se guarda. `GET /clients/{id}/credit` devuelve `credited`, `applied` y `available`.	Igual criterio que el saldo de facturas: no hay totales que se puedan desincronizar.
16	**Anulación bloqueada si el crédito ya se usó**: anular un pago con excedente anula también su crédito, salvo que dejar de contarlo deje el disponible negativo (409 `credit_in_use`, con el disponible en el mensaje). Entonces primero hay que anular las imputaciones (`saldo_a_favor`) en las otras facturas. Lo mismo vale para anular un cobro a cuenta (`POST /credits/{id}/void`, solo administrador).	Nunca queda un crédito negativo ni una factura pagada con plata que no existe.
17	El crédito de un excedente **solo se anula desde su pago** (409 `credit_from_payment` si se intenta desde `/credits`). Anular una imputación `saldo_a_favor` devuelve el importe al disponible.	El excedente y lo imputado forman un único cobro: se anulan juntos.
18	Para reportes de caja: **cobrado** = pagos vigentes con método distinto de `saldo_a_favor` + créditos vigentes. Los pagos `saldo_a_favor` no son dinero nuevo.	Evita contar dos veces el mismo dinero cuando se haga el dashboard de cobranzas.

Transversales

#	Decisión	Motivo
19	**Idempotencia**: header opcional `Idempotency-Key` (UUID) en `POST /payments`, `POST /credits` y `POST /credits/apply`. Un reintento con la misma clave y los mismos datos devuelve lo ya registrado con **200** y `Idempotent-Replayed: true` (el alta nueva da 201). La misma clave con otros datos da 409 `idempotency_key_reused`. El reintento se reconoce **antes** de las reglas de saldo, así que repetir el pago que saldó la factura no da error. El escritorio la va a mandar siempre.	Evita cobros duplicados por doble clic o por reintentos tras un corte de red. Dos envíos simultáneos con la misma clave quedan serializados por los bloqueos de #4, y la restricción `UNIQUE` cubre el resto.
20	Las respuestas de pago incluyen `invoice_balance` e `invoice_status` actuales; las de crédito, `client_available`.	Escritorio y dashboard muestran los saldos nuevos sin otra consulta.
21	**Auditoría** (HU09) en la misma transacción, extendiendo la del Sprint 1 a pagos y créditos. Pago `create`: factura, recibido, imputado, excedente, medio, referencia y `balance_before`/`balance_after`. Crédito `create` (excedente o a cuenta) y `void`. Pago `void`: importe, excedente anulado y motivo.	§3.
22	Los listados excluyen lo anulado salvo `include_void=true`. `/payments` filtra por `invoice_id`, `client_id`, `date_from/to` y `method` (incluye `saldo_a_favor`); `/credits` filtra por `client_id`.	Mismo criterio que las facturas anuladas (ADR-0002 #11).
23	`paid_subquery` (suma de pagos vigentes) pasa a ser pública en `services/invoices.py` y la reutiliza el servicio de pagos.	Una sola definición de "lo cobrado".

Errores nuevos

Código	HTTP	Cuándo
`amount_exceeds_balance`	409	El importe supera el saldo de la factura (sin `allow_credit`, o en una imputación)
`invoice_already_paid`	409	La factura ya tiene saldo 0
`invoice_void`	409	La factura está anulada (también existía para editar facturas)
`payment_already_void`	409	El pago ya estaba anulado
`payment_not_found`	404	Pago inexistente
`idempotency_key_reused`	409	La `Idempotency-Key` ya se usó con otros datos
`insufficient_credit`	409	El cliente no tiene saldo a favor suficiente para imputar
`credit_in_use`	409	Anular dejaría el disponible negativo: hay imputaciones que anular antes
`credit_from_payment`	409	Se intentó anular desde `/credits` el excedente de un pago
`credit_already_void`	409	El cobro a cuenta ya estaba anulado
`credit_not_found`	404	Saldo a favor inexistente

Pendiente para próximos sprints

- Versionar la especificación MVP v1.0 completa en `docs/requirements/` y revisar este ADR contra ella.
- **Devolución** de saldo a favor (reintegro al cliente): hoy solo se puede anular un cobro a cuenta.
- Pantallas de escritorio y dashboard que consuman `/payments` y `/credits` (cobranzas del día, saldo a favor por cliente), aplicando la regla #18.